Concurrency and Resilience Patterns - Cheat Sheet

Three closely-related patterns that show up in every production service: race conditions and deadlocks (when threads collide), retry strategies (when calls fail), and timeout handling (when things take too long). Each is a deep topic on its own.

Race conditions

A race condition exists when the outcome of a computation depends on the order or timing of uncontrolled events. Two threads do something, and the result depends on who gets there first.

Common forms

Read-modify-write. The classic.

```
private int count = 0;
public void increment() {
    count++; // 3 operations: read, add, write
}
```

If two threads call `increment()` at the same time, both can read 0, both write 1. One increment lost.

Check-then-act. The other classic.

```
if (!map.containsKey(key)) {
    map.put(key, value); // racy: another thread may have inserted
}
```

The check passes for both threads. Both call `put`. The last write wins.

Lazy initialization. Races on construction.

```
private Resource resource;
public Resource get() {
    if (resource == null) {
        resource = new Resource(); // both threads may create
    }
    return resource;
}
```

Detection

- Java Flight Recorder + JMC: profile contention and lock events.
- jcstress: the Java Concurrency Stress Test framework. Writes tests that deliberately probe for races.

- Stress tests with many threads: run the suspicious code path concurrently for a long time. Races often surface only under load.
- Code review: scan for shared mutable state without synchronization.

Java doesn't have a native ThreadSanitizer like C/C++ has, but FindBugs/SpotBugs picks up some common cases.

Prevention

The general strategies, ordered by preference:

1. Don't share. Use thread-local state. No sharing, no race.
2. Don't mutate. Immutable objects can be shared freely.
3. Use atomic operations. `AtomicInteger`, `AtomicReference`, compare-and-swap.
4. Use concurrent collections. `ConcurrentHashMap.computeIfAbsent(...)` in place of check-then-act.
5. Synchronize. `synchronized` blocks, `ReentrantLock`, `ReadWriteLock`.
6. Use higher-level constructs. `BlockingQueue`, `CompletableFuture`, channels.

Fixed version of the lazy init race:

```
private final AtomicReference<Resource> resource = new AtomicReference<>();
public Resource get() {
    Resource r = resource.get();
    if (r == null) {
        r = new Resource();
        if (!resource.compareAndSet(null, r)) {
            r = resource.get(); // another thread won, use their value
        }
    }
    return r;
}
```

Or simpler with a memoized supplier:

```
private final Supplier<Resource> resource = Suppliers.memoize(Resource::new); //
        Guava
```

Deadlocks

Two or more threads each hold a lock that the other needs. Both wait forever.

The four necessary conditions

For a deadlock to happen, all four must be true (the Coffman conditions):

1. Mutual exclusion: resources can only be held by one thread at a time.

2. Hold and wait: a thread holds one resource and waits for another.
3. No preemption: a resource can't be forcibly taken away.
4. Circular wait: thread A waits for B, B waits for A (or a longer cycle).

Break any one of the four and deadlock can't happen.

Classic example

```
// Thread 1
synchronized (lockA) {
    Thread.sleep(100);
    synchronized (lockB) {
        // ...
    }
}
// Thread 2
synchronized (lockB) {
    Thread.sleep(100);
    synchronized (lockA) {
        // ...
    }
}
```

Both threads end up holding one lock and waiting on the other. Forever.

Detection

- Thread dump (`jstack <pid>`): shows what each thread holds and waits for. The JVM detects classic deadlocks and prints them at the top of the dump.
- JConsole / VisualVM: GUI tools that show deadlock state in real time.
- Profilers: can reveal contention even when not deadlocked.

A thread dump excerpt that screams deadlock:

```
"Thread-1" waiting to lock monitor 0x00007f8a (object 0x000000076b3, a java.lang.Object)
which is held by "Thread-2"
"Thread-2" waiting to lock monitor 0x00007f8b (object 0x000000076b4, a java.lang.Object)
which is held by "Thread-1"
```

Prevention strategies

1. Lock ordering. Always acquire locks in the same order across threads.

```
// Order by identity hash code so all threads agree
Object first = System.identityHashCode(lockA) < System.identityHashCode(lockB) ?
        lockA : lockB;
Object second = first == lockA ? lockB : lockA;
synchronized (first) {
    synchronized (second) {
        // ...
    }
}
```

2. Lock timeouts. Use `tryLock` with a timeout. Fail fast instead of waiting forever.

```
if (lockA.tryLock(1, TimeUnit.SECONDS)) {
    try {
        if (lockB.tryLock(1, TimeUnit.SECONDS)) {
            try {
                // critical section
            } finally {
                lockB.unlock();
            }
        }
    } finally {
        lockA.unlock();
    }
}
```

3. Avoid nested locks. The simplest deadlock prevention. If you only ever hold one lock at a time, you can't deadlock.
4. Use lock-free data structures. `ConcurrentHashMap`, `AtomicReference`, `LongAdder`. No locks, no deadlock.
5. Single-threaded executors for sensitive paths. Run conflicting work on the same thread. No locks needed at all.

Related: livelock and starvation

Livelock: threads aren't blocked, but they keep responding to each other and make no progress. Two people sidestepping in a hallway.

Starvation: a thread never gets the lock because greedier or higher-priority threads keep grabbing it first. Fairness-mode locks (`new ReentrantLock(true)`) help.

Retry strategies

When a call fails, sometimes retrying is the right move. Often it isn't. Get the rules right.

When to retry

Situation	Retry?
Network connection failure	Yes
HTTP 503 Service Unavailable	Yes
HTTP 429 Too Many Requests	Yes, after backoff
HTTP 504 Gateway Timeout	Yes (if idempotent)
gRPC UNAVAILABLE	Yes
gRPC DEADLINE_EXCEEDED	Yes (if idempotent)
HTTP 400 Bad Request	No
HTTP 401 Unauthorized	No (refresh token first if expired)
HTTP 403 Forbidden	No
HTTP 404 Not Found	No
Validation failure	No
Out of memory	No

The rule: retry only when the call might succeed if attempted again, and only when re-execution is safe (idempotent, or protected by an idempotency key).

Backoff strategies

Constant delay. Wait the same amount each time. Simple but creates retry storms.

```
retry 1: wait 200ms
retry 2: wait 200ms
retry 3: wait 200ms
```

Linear backoff. Increase by a fixed amount.

```
retry 1: wait 200ms
retry 2: wait 400ms
retry 3: wait 600ms
```

Exponential backoff. Double each time. Recommended default.

```
retry 1: wait 200ms
retry 2: wait 400ms
retry 3: wait 800ms
retry 4: wait 1600ms (capped at some max)
```

Jitter

Without jitter, all clients retrying after a service blip hit it at the same instant. The thundering herd finishes the service off.

Full jitter: random between 0 and the current backoff.

```
long base = (long) (200 * Math.pow(2, attempt));
long delay = ThreadLocalRandom.current().nextLong(base);
```

Equal jitter: half the base plus a random offset.

```
long base = (long) (200 * Math.pow(2, attempt));
long delay = base / 2 + ThreadLocalRandom.current().nextLong(base / 2);
```

AWS architecture guides recommend full jitter for most cases.

Idempotency

A retry is only safe if running the operation twice produces the same result as running it once.

For non-idempotent operations (creating a resource, charging a card), use an idempotency key:

```
POST /orders
Idempotency-Key: 8f3a-2bc4-...
```

The server stores the response keyed by the idempotency key. Retries with the same key return the stored response without re-doing the work.

Retry budgets

A retry is a load amplifier. If every request retries up to 3 times, a service hiccup turns 1x load into 4x load and makes recovery harder.

Defenses:

- Cap retries per call. 3 is usually enough.
- Set a global budget. “Across all clients, at most 10% of traffic can be retries.” Reject the rest.
- Skip retries when a circuit breaker is open. No point if the dependency is known sick.
- Add jitter. Spread retry traffic over time.

Libraries

Use a library. Don't roll your own.

- Resilience4j (modern Java): clean API, integrates with Spring.
- Spring Retry: simple, decorator-based.
- Failsafe: lightweight, framework-neutral.
- AWS SDK / Google Cloud SDK: client libraries with built-in retry strategies tuned for their services.

Resilience4j example:

```
RetryConfig config = RetryConfig.custom()
    .maxAttempts(3)
    .waitDuration(Duration.ofMillis(500))
    .retryOnException(e -> e instanceof IOException)
    .build();
Retry retry = Retry.of("user-service", config);
Supplier<User> decorated = Retry.decorateSupplier(retry, () -> userClient.find(id));
User user = decorated.get();
```

Timeout handling

Every external call needs a deadline. Without one, a slow dependency holds your thread, and enough slow calls exhaust the pool. Then your service stops responding.

Types of timeouts

Timeout	What it bounds
Connect timeout	Time to establish a TCP connection
Read / socket timeout	Time between bytes during a read
Request / overall timeout	Total time the request can take
Idle timeout	Time a connection sits unused before being closed

You usually want both a connect timeout and a request timeout. The request timeout is the safety net.

```
HttpClient client = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(2))
    .build();
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/users/42"))
    .timeout(Duration.ofSeconds(5))
    .build();
```

Picking values

A good timeout sits a little above the legitimate worst case for that call. If p99 latency is 200ms and the worst real success is 800ms, set the timeout around 1-2 seconds.

Rules of thumb:

- Look at p99 latency in production. Set the timeout 2-3x above it.
- Connect timeouts should be short (1-2s). A new TCP handshake shouldn't take longer.
- Request timeouts for fast services (cache, in-cluster RPC): 100ms to a few seconds.
- Request timeouts for slow services (3rd party API, ML inference): 5-30 seconds. But never "no timeout".
- Background jobs can have longer timeouts (minutes) since they don't block users.

Deadline propagation

In a service chain (A -> B -> C), the deadline should flow with the request. If A has a 5-second budget and B's call took 1s, B should give C at most 4 seconds.

```
// gRPC propagates deadlines automatically
stub.withDeadlineAfter(2, TimeUnit.SECONDS).getUser(request);
```

For HTTP, pass remaining time via a header (`X-Deadline-Ms` , or `traceparent` with timing info).

Without propagation, downstream services keep working on a request the caller has already given up on.

Handling timeout failures

A timeout doesn't mean the work didn't happen. The server may have completed the operation just after the client gave up.

Implications:

- Idempotent operations: retry safely.
- Non-idempotent operations: use idempotency keys, or accept that retrying might double-create.
- Background reconciliation: for critical operations, run a reconciler that detects and fixes inconsistencies caused by partial failures.

Common pitfalls

- No timeout at all. Default `HttpURLConnection` , old Apache HttpClient defaults, several JDBC drivers. Many leave timeouts at infinite. Always configure them.
- Connect timeout but no read timeout. Connection succeeds, server starts responding, then a network blip leaves the read hanging. Set both.
- Caller timeout shorter than the callee's. Client gives up, but the server keeps working. Wasted resources and possible inconsistencies.

- All timeouts identical. A 5-second timeout for a cache lookup is wasteful. A 5-second timeout for an ML model is unrealistic. Tune per dependency.
- Timeouts that don't actually fire. Some libraries claim to support timeouts but only check between operations. A blocked socket read may never check.

Combining them: the layered defense

In production, you compose these patterns:

```
Timeout
└─ if the call doesn't complete in N seconds, abort
Retry (with backoff + jitter)
└─ if the call fails with a retryable error, try again
Circuit breaker
└─ if too many failures recently, stop trying for a while
Bulkhead
└─ limit concurrent calls to this dependency
Fallback
└─ if everything above fails, return a default
```

Order matters. The circuit breaker wraps the retry. The retry wraps the timeout. The bulkhead caps concurrency around all of it. The fallback catches anything that escapes.

Resilience4j composition:

```
Supplier<User> withFallback = Decorators.ofSupplier(() -> userClient.find(id))
    .withCircuitBreaker(circuitBreaker)
    .withRetry(retry)
    .withBulkhead(bulkhead)
    .withFallback(List.of(Throwable.class), e -> cachedUser(id))
    .decorate();
User user = withFallback.get();
```

Best practices

- Use immutability and concurrent collections first. They eliminate whole classes of races.
- Make locks short and predictable. Long critical sections cause contention and deadlocks.
- Acquire locks in a consistent order across threads. The simplest deadlock prevention.
- Use `tryLock` with a timeout when you can't enforce ordering.
- Set timeouts on every external call. Connect, read, and overall request.
- Retry only idempotent operations or pass an idempotency key.
- Always add jitter to retry backoff. Avoid synchronized retry storms.
- Cap retries per call and globally. Don't let retries amplify outages.
- Treat slow calls as failures. A response that doesn't come in time is just as bad as an error.

- Layer the patterns. Timeout + retry + circuit breaker + fallback. Each handles a different shape of failure.

Common gotchas

- `volatile` isn't a lock. It only fixes visibility and ordering. Atomicity isn't covered, so `count++` on a volatile field is still racy.
- `Collections.synchronizedMap(...)` doesn't make compound operations atomic. `if (!map.containsKey(k)) map.put(k, v)` is still racy. Use `ConcurrentHashMap.putIfAbsent` instead.
- Java's `synchronized` is reentrant. Other primitives may not allow re-entry. Check before swapping.
- Release locks in `finally`. Otherwise a thrown exception leaves the lock held forever.
- Sleep doesn't release locks. `Thread.sleep()` inside a `synchronized` block keeps the lock held the whole time.
- Deadlocks across services. Service A holds DB row X and calls B, which holds row Y and calls A. Distributed deadlock. Use timeouts and avoid cross-service locking.
- Retry on `DEADLINE_EXCEEDED` without considering idempotency. Easy way to double-charge a customer.
- Caller timeout shorter than the retry budget. A 5s caller timeout with 3 retries of 3s each can hit 9 seconds. Math your retries against your timeout.
- Connection pool exhaustion from timeouts. Slow upstream + retries + no bulkhead = thread pool full of stuck calls.
- Library defaults aren't safe defaults. Many HTTP and JDBC libraries default to infinite timeouts. Audit and set them explicitly.

Quick reference

Concept	Key fact
Race condition	Result depends on thread timing
Read-modify-write	Classic race (`count++` is not atomic)
Check-then-act	The other classic race
Deadlock	Two threads each hold a lock the other needs
Coffman conditions	Mutual exclusion, hold-and-wait, no preemption, circular wait
Lock ordering	Always acquire in the same order across threads
`tryLock`	Acquire with timeout, fail fast instead of deadlocking
Retry	Only on transient failures and idempotent operations
Exponential backoff + jitter	Recommended default for retries
Retry budget	Cap retries to prevent load amplification
Idempotency key	Makes non-idempotent operations safe to retry
Connect timeout	Time to set up the TCP connection
Request timeout	Total time the request can take
Deadline propagation	Pass remaining budget through the call chain
Layered defense	Timeout + retry + circuit breaker + fallback